

## 33. Полиморфизмы. Вычисление типов унификацией выражений типа.

### 1. Понятие полиморфизма

**Определение:** Структурный элемент программы (выражение, инструкция, функция, класс и т. д.) называется **полиморфным**, если он может однотипно выполняться с аргументами различных типов.

- **Встроенные полиморфные операторы:** В промышленных языках это операторы индексирования массивов, вызова функций или работы с указателями.
- **Особенности реализации:** Возможность применения такого оператора обычно определяется не конкретным типом аргумента, а только «меткой» корневого узла соответствующего выражения типа.

### 2. Примеры полиморфных операторов

#### Разыменование указателя (Правило 16)

Оператор, возвращающий значение по указателю на него (разыменование), задается выражением типа  $ptr(t) \rightarrow t$ .

- Если выражение  $E_1$  имеет тип  $ptr(t)$ , то тип результата  $E.type = t$ .
- В противном случае возникает ошибка `type_error`.
- **Квантор всеобщности:** С точки зрения матлогики, каждая переменная типа  $(t)$  в таких выражениях связана квантором всеобщности  $(\forall t)$ .

#### Индексирование массива (Правило 17)

Оператор индексирования задается выражением типа  $(array(N, t) \times int) \rightarrow t$ .

- Если выражение  $E_1$  имеет тип массива с элементами типа  $t$ , а индекс  $E_2$  имеет тип  $int$ , то результат  $E_1[E_2]$  получает тип  $t$ .
- Переменная  $t$  может быть любым типом, что и делает оператор полиморфным.

### 3. Унификация выражений типа

**Унификация** — это процедура уточнения типов путем выполнения замен переменных типа на конкретные выражения типа.

- **Процесс:** Имея два (или более) выражения типа, мы выполняем в них одновременно ряд замен для **всех вхождений** одной переменной во все исходные выражения.
- **Цель:** Сделать так, чтобы полученные примеры исходных выражений совпали.
- **Результат:** \* Если набор замен существует, типы называются **унифицируемыми**, а набор замен — **унификатором**.
  - Если унификатора не существует, в программе содержится **ошибка типов**.
  - В качестве итогового типа берется их **общий пример** (наиболее общий унификатор).

### 4. Разбор сложного примера: Вычисление типа $x[x[y] \uparrow] \uparrow$

На примере выражения  $x[x[y] \uparrow] \uparrow$  показано, как работает алгоритм на ориентированном ациклическом графе (DAG):

#### Шаги вычисления:

1. **Начальные данные:**
  - $x$  имеет тип  $\alpha$ .
  - $y$  имеет тип  $\beta$ .

2. **Анализ внутреннего выражения  $x[y]$ :**
- Чтобы применить оператор индексирования к  $x$ , его тип  $\alpha$  должен быть унифицирован с типом  $array(N, \gamma)$ .
  - Тип  $y$  ( $\beta$ ) должен быть унифицирован с  $int$ . Значит,  $\beta = int$ .
3. **Анализ разыменования  $\uparrow$ :**
- К результату  $x[y]$  (тип  $\gamma$ ) применяется оператор `deref`. Значит,  $\gamma$  должен иметь вид  $ptr(t)$ .
  - Таким образом, уточняется тип  $x$ : теперь  $\alpha = array(N, ptr(t))$ .
4. **Анализ внешнего индексирования:**
- Внешний оператор индексирования снова работает с  $x$  (тип  $\alpha$ ) и индексом (результат  $x[y] \uparrow$  имеет тип  $t$ ).
  - Для успешной унификации индекс должен быть  $int$ . Следовательно,  $t = int$ .
5. **Итоговый вывод:**
- Тип переменной  $x$  уточнен как  $array(N, ptr(int))$ .
  - Итоговый тип всего выражения ( $\lambda$ ) вычисляется как  $int$ .

5. Дополнительные сведения

- Неявные преобразования:** В реальных процессорах нельзя просто сложить целое и действительное число, так как у них разное машинное представление. Компиляторы высокого уровня берут на себя задачу автоматического (неявного) преобразования типов для удобства программиста.
- Алгоритм:** Для унификации нециклических выражений используется стандартный алгоритм унификации термов из математической логики.

Расшифровка символов и обозначений типов

| Символ / Запись                | Что означает           | Пояснение                                                                                                                       |
|--------------------------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| $t, s, \alpha, \beta, \gamma$  | Переменные типа        | Обозначают неизвестный тип, который нужно вычислить. Например, $t$ может оказаться или <code>char</code> в процессе унификации. |
| $ptr(t)$                       | Pointer (Указатель)    | Обозначает тип «указатель на тип $t$ ». Используется для операций разыменования.                                                |
| $array(N, t)$                  | Массив                 | Обозначает массив из $N$ элементов типа $t$ .                                                                                   |
| $\uparrow$ (стрелка вверх)     | Разыменование          | Оператор получения значения, на которое указывает указатель. В коде это часто <code>*p</code>                                   |
| $[]$                           | Индексирование         | Оператор доступа к элементу массива по индексу.                                                                                 |
| $\rightarrow$ (стрелка вправо) | Отображение (Функция)  | Описывает сигнатуру оператора. Запись $A \rightarrow B$ значит: «принимает тип $A$ , возвращ                                    |
| $\times$                       | Декартово произведение | Обозначает, что оператор принимает несколько аргументов. Например, (Тип1 $\times$ Ти                                            |
| <code>int</code>               | Целое число            | Базовый атомарный тип данных.                                                                                                   |
| <code>type_error</code>        | Ошибка типа            | Специальное значение, которое возвращается, если типы несовместимы (унификаци                                                   |

Подробный разбор концепций

1. Что такое `ptr(t)` ?

Это описание типа данных в памяти. Если мы говорим, что переменная имеет тип `ptr(int)`, значит, в ней лежит адрес, по которому находится целое число. Полиморфный оператор разыменования ( $\uparrow$ ) работает так: он «снимает» оболочку `ptr`. Если на вход подан `ptr(t)`, на выходе мы получаем чистый `t`.

2. Зачем нужны греческие буквы ( $\alpha, \beta, \gamma, \delta$ )?

В примере с графом (DAG) они используются как временные «имена-заглушки» для узлов:

- $\alpha$  — это тип идентификатора  $x$ .
- $\beta$  — это тип идентификатора  $y$ .
- Когда анализатор видит  $x[y]$ , он понимает, что  $\alpha$  обязан быть массивом, а  $\beta$  — целым числом. Так происходит «уточнение» через унификацию.

### 3. Как читать запись $(array(N, t) \times int) \rightarrow t$ ?

Это математическое описание правила индексации массива:

1. Берем массив элементов типа  $t$  ( $array(N, t)$ ).
2. Берем целое число ( $int$ ).
3. Применяем оператор  $[]$ .
4. **Результат:** получаем один элемент того самого типа  $t$ .

### 4. Что такое Унификация (простыми словами)?

Это процесс решения уравнения.

- У нас есть шаблон: Указатель на что-то ( $ptr(t)$ ).
- У нас есть факт: Указатель на целое ( $ptr(int)$ ).
- **Унификация:** мы сравниваем их и делаем вывод, что «что-то» ( $t$ ) — это точно  $int$ . Если бы мы пытались сравнить  $ptr(int)$  и  $array(10, char)$ , компилятор выдал бы `type_error`, так как они не «унифицируются» (не похожи друг на друга по структуре).

Такие жесткие условия на типы аргументов выражений и инструкций, как в грамматике для проверки типов из лекции 24, не встречаются в реальных языках программирования высокого уровня, хотя на машинном уровне это необходимо. Процессор не может сложить целое число с действительным: действительные числа имеют иное внутреннее представление, нежели целые, а для их сложения используется другая машинная инструкция. Требуется преобразование типов. Общая концепция работы с типами: в обращении с типами допускаются все «вольности», которые полезны для программистов с точки зрения создателей языка, при условии, что компилятор в состоянии распознать все разрешенные «несоответствия» типов и выполнить необходимые преобразования автоматически (*неявные* преобразования), в отличие от *явных* — записанных в программе.

## Определение

Рассмотрим работу с выражениями типа, содержащими переменные.

### Определение

Структурный элемент программы (выражение, инструкция, функция, класс, метод и т. п.) называется *полиморфным*, если он может однотипно выполняться с аргументами различных типов.

В любом промышленном языке программирования есть встроенные полиморфные операторы, например, для индексирования массивов, вызова функций или работы с указателями. Возможность применения такого оператора обычно определяется не типом аргумента, а только меткой корневого узла соответствующего выражения типа.

## Пример

Рассмотрим еще раз атрибутивную грамматику для проверки типов (лекция 24).

|    |                          |                                                                                                                                                       |
|----|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 17 | $E \rightarrow E_1[E_2]$ | $E.type = \begin{cases} t, & \text{если } E_2.type = \text{int}, \\ & E_1.type = \text{array}(N, t) \\ \text{type\_error} & \text{иначе} \end{cases}$ |
|----|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|

Правило 17 показывает, как работает встроенный оператор индексирования массива: если выражение  $E_1$  имеет тип массива с элементами типа  $t$ , а  $E_2$  имеет тип  $\text{int}$ , то выражение  $E_1[E_2]$  получает тип  $t$ . Поскольку тип  $t$ , вообще говоря, может быть любым, естественно рассматривать  $t$  как переменную типа; оператор индексирования задается выражением типа  $(\text{array}(N, t) \times \text{int}) \rightarrow t$ . Заметим, что наличие переменных типа – отличительная черта выражений типа с полиморфными операторами.



## Пример

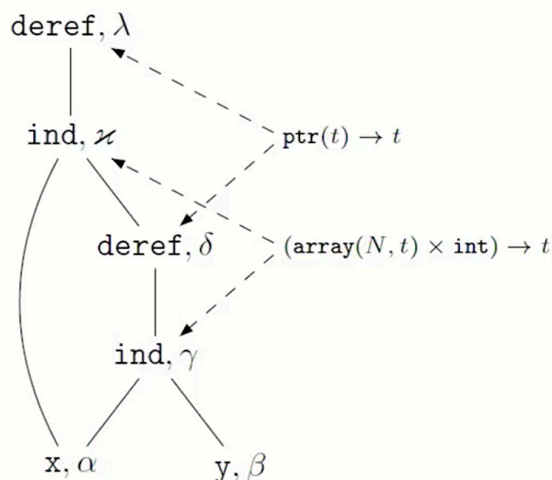
|    |                              |                                                                                                                      |
|----|------------------------------|----------------------------------------------------------------------------------------------------------------------|
| 16 | $E \rightarrow E_1 \uparrow$ | $E.type = \begin{cases} t, & \text{если } E_1.type = \text{ptr}(t) \\ \text{type\_error} & \text{иначе} \end{cases}$ |
|----|------------------------------|----------------------------------------------------------------------------------------------------------------------|

Правило 16 определяет, как работает полиморфный оператор, возвращающий значение по указателю на него (этот оператор иногда называют *разыменованием указателя*). Разыменование задается выражением типа  $\text{ptr}(t) \rightarrow t$ .

Частные случаи, или *примеры*, выражения типа с переменными получаются при замене переменных выражениями типа (все вхождения одной и той же переменной должны заменяться одинаково). Последнее выражение типа имеет примеры  $\text{ptr}(\text{char}) \rightarrow \text{char}$ ,  $\text{ptr}(\text{ptr}(t)) \rightarrow \text{ptr}(t)$  и т. п.

*С точки зрения матлогики каждая переменная типа в выражениях с полиморфизмами связана квантором всеобщности.*

## Вычисление типа выражения



$x[x[y] \uparrow] \uparrow$

При рассмотрении второго оператора индексирования мы получаем  $\delta = \text{int}$  и  $\alpha = \text{array}(N, \kappa)$ . Но в этот момент мы уже знаем, что  $\alpha = \text{array}(N, \gamma) = \text{array}(N, \text{ptr}(\text{int}))$ ; следовательно,  $\kappa = \text{ptr}(\text{int})$ . Наконец, рассмотрев оператор `deref` в корне дага, мы получаем

$\lambda = \text{int}$ . Таким образом, тип выражения вычислен, а попутно установлены типы входящих в него идентификаторов (кроме размера массива  $x$ ).

# Унификация

Процедура, которая применялась в каждом узле дага в последнем примере для уточнения типов выражений, называется *унификацией* выражений типа. Более точно, процедура унификации такова: имея два (или более) выражений типа, мы выполняем в них одновременно ряд замен переменных на выражения типа (для *всех* вхождений одной переменной во *все* исходные выражения выполняется *одна и та же* замена) так, чтобы полученные примеры исходных выражений типа совпали.

Если такой набор замен существует, то выражения типов называются *унифицируемыми*, а любой подходящий набор замен – *унификатором*. Заметим, что выражения типа без переменных унифицируемы тогда и только тогда, когда они эквивалентны.

## Унифицируемость выражений

Пусть для фрагмента программы вычислены разными способами два выражения типа; если они унифицируемы, то в качестве выражения типа для этого фрагмента берется их общий пример; если они неунифицируемы, то фрагмент содержит ошибку типов.

Например, при анализе верхнего из узлов с меткой `ind` мы унифицировали выражения `array(N, x)` и `array(N, ptr(int))` для типа идентификатора `x` заменой `x = ptr(int)`. Если бы унификатора не существовало, то мы бы обнаружили, что использование идентификатора `x` некорректно с точки зрения типов.

Для унификации нециклических выражений типа можно применить алгоритм унификации термов из стандартного курса математической логики.